

C/C++静态代码检查器自动生成软件 用户手册

一、背景

在软件开发领域，编程规则的制定与遵循对于提升代码的安全性与可靠性、降低潜在漏洞风险具有重要作用。为自动验证代码对编程规则的符合性，静态代码检查工具被广泛采用，其通过内置的规则检查器对源代码进行静态分析和合规性审查，有效辅助开发人员识别违规行为。

随着互联网技术的快速发展和软件复杂度的不断提升，企业与用户对软件质量与安全的要求日益严格，大量新的编程规则不断涌现以应对各类新型安全挑战。然而，现有静态代码检查工具所内置的规则检查器数量有限，更新速度较慢，难以全面覆盖持续扩展的检测需求，导致工具能力与实际应用之间出现显著差距。尽管多数工具允许用户自定义编程规则检查器，但其开发过程通常涉及检测需求的明确、违规场景的梳理以及特定框架下的代码实现，对普通用户而言存在较高的技术门槛和时间成本。

针对上述问题，C/C++静态代码检查器自动生成软件 **AutoChecker** 应运而生。该软件充分利用大型语言模型（LLM）强大的代码生成能力，能够根据用户指定的编程规则自动生成相应的 C/C++规则检查器代码，显著降低用户编写自定义检查器的难度和工作量，有效弥补了现有工具在规则覆盖和灵活定制方面的不足。

二、系统目标

AutoChecker 的目标是生成定制化的代码检查器，可以直接应用于 **Clang-Tidy** 的环境中，执行相关源代码检查。软件以用户自定义的编程规则和相关违例代码作为输入，充分利用 LLM 的代码生成能力，生成高质量的代码检查器。

三、用户的软件环境：

操作系统：Ubuntu 22.04

开发及运行时环境：Python3.10

依赖管理：Anaconda(用于安装依赖和运行命令)

外部工具：

- (1) clang(建议使用 LLVM release/17.x 版本)
- (2) cmake (版本 ≥ 3.10)
- (3) Docker (建议使用 docker 28.1.1 版本)

四、安装与构建

1. 构建镜像

把源代码克隆到本地：

```
git clone https://github.com/SQUARE-RG/AutoChecker.git
```

进入软件根目录，如图 1 所示：

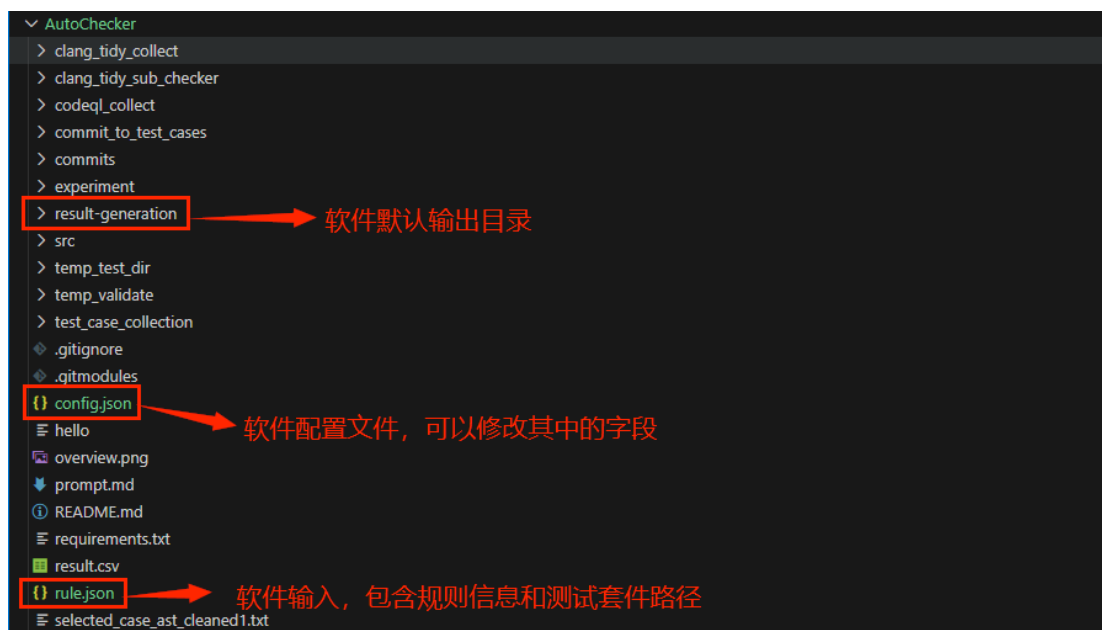


图 1 AutoChecker 目录树

接着在软件根目录下克隆 llvm 仓库

```
git clone https://github.com/llvm/llvm-project.git
```

软件根目录下有 Dockerfile 文件，接着使用 docker 构建镜像,成功后如图 2 所示：

```
docker build -t autochecker:1.0 .
```

镜像构建过程中涉及构建 Python 环境，编译 Clang-tidy 等操作。具体来说，首先在 ubuntu 22.04 中安装 miniconda，然后创建一个虚拟环境用来安装所有的依赖，接着使用 cmake 工具编译 Clang-tidy，准备好工具链环境，在 Clang-tidy 的源代码中新增加了一个代码检查器（仅仅用于测试环境是否可用），编译后执行检测，检查当前的 Clang-tidy 工具链是否可用。整个过程需要大概 10 分钟左右。

```

=> [internal] load metadata for docker.io/library/ubuntu:22.04
0.0s
=> [internal] load .dockerignore
0.1s
=> => transferring context: 2B
0.0s
=> [internal] load build context
9.7s
=> => transferring context: 17.06MB
9.1s
=> [ 1/11] FROM docker.io/library/ubuntu:22.04
0.0s
=> CACHED [ 2/11] RUN apt-get update && apt-get install -y --no-install-recommends git wget curl bzip2 ca-certificates build-essential 0.0s
=> CACHED [ 3/11] WORKDIR /root
0.0s
=> [ 4/11] COPY code_check /root/code_check
13.5s
=> [ 5/11] RUN echo "=== 查看 /root 目录 ===" && ls -la /root && echo "=== 查看 /root/code_check 是否存在 ===" && if [ -d "/root/code_check" ]; then 2.5s1] RUN cd /root/code_check && git checkout web-release && true 2.8s
=> [ 6/11] RUN cd /root/code_check && git checkout web-release && true
2.8s
=> [ 7/11] COPY llvm-project /root/code_check/llvm-project
86.0s
=> [ 8/11] RUN find /root/code_check/llvm-project -type f | head -10
1.9s
=> [ 9/11] RUN cd /root/code_check && chmod +x scripts/install.sh && bash scripts/install.sh
1907.4s
=> [10/11] RUN /root/anaconda3/bin/conda create -y -n code_heck python=3.10 && /root/anaconda3/envs/code_heck/bin/pip install -no-cache-dir modelscope && /r 462.3s
=> [11/11] WORKDIR /root/code_check
1.7s
=> exporting to image
636.2s
=> => exporting layers
635.7s
=> => writing image sha256:ada2b993af45595cbc24c1c289caf84b89b5bcd2a5ced47baa568e21cb1ed84d
0.0s
=> => naming to docker.io/library/autochecker:1.0

```

图 2 AutoChecker 镜像构建成功

2. 构建容器

在上一步构建的镜像基础上构建容器：

```
docker run -it --name autochecker-container autochecker:1.0 /bin/bash
```

直接进入容器进入交互模式。

3. LLM 配置

进入容器后，默认进入软件的根目录，创建一个.env 文件，填写以下 API_KEY, MODEL 和 BASE_URL 三个字段，下面给出一个示例：

```
API_KEY= XXXXX/
MODEL = Deepseek
BASE_URL= https://api.deepseek.com
```

至此，软件部署完成。

五、工具使用流程

5.1 输入与输出

软件输入：

- (1) 规则和测试用例信息，可以集成在 `rule.json` 中。

软件输出：

- (1) 代码检查器，通常包含一个 `.cpp` 文件和 `.h` 文件，位于指定或者默认的输出目录下。
- (2) 检测结果报告，通常包含一个 `json` 文件，位于指定或者默认的输出目录下。

5.2 配置定制的检测规则及测试用例

修改软件根目录的 `rule.json` 文件，这个 `json` 文件包含规则信息和对应的测试用例路径，可以作为软件的输入，如图 1 所示。

```
1 {
2   "data": {
3     "ucassaat": [
4       {
5         "title": "clang-tidy - declare-anonymous-struct",
6         "main_title": "declare-anonymous-struct",
7         "description": "This rule prohibits the inclusion of anonymous structs (i.e., nested struct types without a variable name) within struct definitions. An anonymous struct refers to a type that is directly embedded inside an outer struct but lacks an explicit variable name identifier. If a nested struct is assigned a specific variable name, it complies with the rule. This rule applies to all levels of struct nesting, including multi-level scenarios, and covers direct members of structs regardless of their scope (global or local).\nScenarios that should be reported include: structs containing directly defined anonymous structs (without a variable name), anonymous unions within structs, multi-level nested structs with anonymous structs at any level, and anonymous structs appearing as members of other structs.\nCorrect scenarios include: nested structs having explicit variable names, structs defined and used normally without any nested anonymous structs, named structs appearing as members within an outer struct, and struct types defined via typedef.\nNote: The term \"anonymous struct\" specifically denotes a nested struct that is defined inline without a named identifier, distinguishing it from standalone unnamed structs which are not covered by this rule.",
8         "category": "ucassaat",
9         "rule_test_path": "/root/code_check/llvm-project/clang-tools-extra/test/clang-tidy/checkers/ucassaat/declare_anonymous_struct"
10      }
11    ]
12  }
13 }
14 }
```

图 3 软件输入 `rule.json` 示例

在输入的 `rule.json` 文件中，`main_title` 字段是必填的规则名称，`title` 字段可选，`description` 字段填写该规则描述，说明该规则的内容和适用范围。`rule_test_path` 字段是测试用例套件的路径，推荐使用绝对路径，而 `category` 字段可选，通常标识这个规则的类型。

测试用例的要求是以 `cpp` 或者 `c` 结尾的代码文件，且每个代码文件必须可以独立编译，保证没有语法或者其他错误。

软件默认输出位置是根目录下的 `result-generation` 目录，这里保存着第一阶段生成的检查器代码和最终生成的检查器代码。Clang-Tidy 的检查器代码包含一个后缀为 `h` 的头文件和一个后缀为 `cpp` 的代码文件。

5.3 配置基本设置项

用户可以根据自己的需求调整配置文件，即软件根目录下的 config.json，需要调整的字段及其说明如表 1 所示。

字段	说明	示例
embedding_model	Embedding 模型的路径	src/retriever/embedding_model/bge-large-en-v1.5
result_dir	软件输出路径	result-generation/
max_round	处理每个测试用例的迭代最大次数	2
max_compiler_trys	修复检查器编译失败的最大次数	2
top_key	检索前 K 个最相关的代码片段	2

表 1 软件配置 config.json 相关字段说明

5.4 启动检查器生成命令

完成上述准备工作后，可以使用如下命令启动软件：

```
python src/main.py --input rule.json
```

```
2025-12-23 07:40:05.704 | INFO | retriever.retrieve_from_ast_api:get_related_ast_api:123 - 开始计算logics的嵌入
2025-12-23 07:40:06.241 | INFO | retriever.retrieve_from_ast_api:get_data:25 - API list already exists at
src/embedding_db/api_list.pt. Loading existing data.
2025-12-23 07:40:06.353 | INFO | generator:generate_checker_with_single_case_and_checker:154 - 相关API上下文检索完成
2025-12-23 07:40:06.355 | INFO | generator:generate_checker_with_single_case_and_checker:168 - 针对负例/root/code_check/llvm-
project/clang-tools-extra/test/clang-
tidy/checkers/ucassaat/no_same_name_as_global_variable/no_same_name_as_global_variable_case_6.cpp开始使用增强逻辑生成checker代码
2025-12-23 07:40:06.356 | INFO | generator:generate_checker_with_query:200 - 使用增强逻辑生成checker代码，尝试第1次
2025-12-23 07:40:53.498 | INFO | generator:checker_augmentation:467 - 增强阶段，开始编译
2025-12-23 07:41:08.920 | INFO | generator:checker_augmentation:493 - 增强阶段，第2轮生成的checker编译成功，下面运行测试用例进行验证
2025-12-23 07:41:09.016 | INFO | generator:checker_augmentation:501 - 增强阶段，测试用例运行完成，warning数量为:0
2025-12-23 07:41:09.016 | INFO | generator:checker_augmentation:445 - 增强阶段，达到最大生成轮数2，停止当前测试用例的生成尝试
2025-12-23 07:41:10.760 | INFO | generator:runAllTestCase:415 - 运行所有测试用例完成，成功通过的测试用例数量:19/20
2025-12-23 07:41:10.761 | INFO | _main_:save_final_checkers:61 - 最终checker已保存到: /root/code_check/result-generation/no-
same-name-as-global-variable/final_checker/
2025-12-23 07:41:10.761 | INFO | _main_:main:174 - 生成了 1 个Checker，规则名: no-same-name-as-global-variable
2025-12-23 07:41:10.761 | INFO | _main_:main:176 - 最终生成的Checker通过的测试用例数量:19/20
2025-12-23 07:41:11.143 | INFO | _main_:main:181 - 已删除Clang仓库中的Checker，规则名: no-same-name-as-global-variable
2025-12-23 07:41:11.143 | INFO | _main_:main:195 - Checker生成完毕，结果已保存
2025-12-23 07:41:11.143 | INFO | _main_:main:197 - 规则 no-same-name-as-global-variable 的Checker生成总共耗时: 315.65 秒
```

图 4 软件运行过程中的输出

5.5 输出目录与结果解读

软件运行结束后，默认的输出目录是软件根目录下的 `result-generation` 文件夹，如图 5 所示：

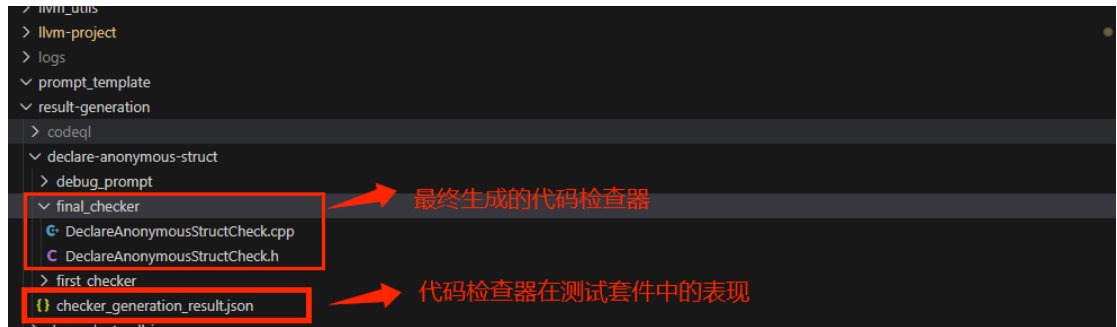


图 5 软件输出目录

软件会为输入的规则在 `result-generation` 目录下单独创建一个文件夹，例如图 3 输入 `rule.json`，其 `main_title` 字段是规则名称 `declare-anonymous-struct`，那么这个规则的生成结果会保存在 `result-generation/declare-anonymous-struct` 目录下，在这个目录下，`debug_prompt` 文件夹保存用于 `debug` 的日志，`first_checker` 文件夹用于保存 LLM 生成的中间结果，`final_checker` 文件夹保存着最终生成的代码检查器，`checker_generation_result.json` 是生成的检查器在测试套件中的检测结果统计。

5.6 获取生成检查器的测试报告

生成的检查器在测试套件中的表现保存在 `checker_generation_result.json` 文件中，内容如图 6 所示。

```
1 {
2   "data": {
3     "ucassaat": [
4       {
5         "title": "clang-tidy - declare-anonymous-struct",
6         "main_title": "declare-anonymous-struct",
7         "description": "This rule prohibits the inclusion of anonymous structs (i.e., nested struct",
8         "category": "ucassaat",
9         "rule_test_path": "declare_anonymous_struct",
10        "negative_case_amount": 10,
11        "positive_case_amount": 10,
12        "issuccess": "True",
13        "performance": "20/20",
14        "success_case_list": [
15          "declare_anonymous_struct/declare_anonymous_struct_case_18.cpp", "declare_anonymous_str",
16          "declare_anonymous_struct/declare_anonymous_struct_case_6.cpp", "declare_anonymous_stru",
17          "declare_anonymous_struct/declare_anonymous_struct_case_4.cpp", "declare_anonymous_stru",
18          "declare_anonymous_struct/declare_anonymous_struct_case_1.cpp", "declare_anonymous_stru",
19          "declare_anonymous_struct/declare_anonymous_struct_case_2.cpp", "declare_anonymous_stru",
20          "declare_anonymous_struct/declare_anonymous_struct_case_10.cpp", "declare_anonymous_str",
21          "declare_anonymous_struct/declare_anonymous_struct_case_16.cpp", "declare_anonymous_str",
22          "declare_anonymous_struct/declare_anonymous_struct_case_7.cpp", "declare_anonymous_stru",
23          "declare_anonymous_struct/declare_anonymous_struct_case_12.cpp", "declare_anonymous_str",
24          "declare_anonymous_struct/declare_anonymous_struct_case_15.cpp"
25        ],
26        "failed_case_list": [],
27        "total_cost": "0.049582",
28        "time": "330.61"
29      }
30    ]
31  }
32 }
```

图 6 软件输出 `checker_generation_result.json` 示例

从输出结果的 performance 字段来看，软件生成的代码检查器通过了所有的测试用例，准确率 100%。total_cost 字段显示了整个迭代过程使用 LLM 的 api 花费约 0.04 元人民币，成本低廉。time 字段记录了整个生产过程花费 330 秒，时间花销也在合理范围内。

5.7 获取生成检查器的最终代码

最终输出的检查器代码通常会保存在默认的输出目录: <AutoChekcer 所在目录>/result-generation/final_checker，图 5 中的 DeclareAnonymousStructCheck.cpp 和 DeclareAnonymousStructCheck.cpp.h 就是最终生成的检查器代码文件，DeclareAnonymousStructCheck.cpp 的内容如图 7 所示。

```
#include "DeclareAnonymousStructCheck.h"
#include "clang/AST/ASTContext.h"
#include "clang/ASTMatchers/ASTMatchFinder.h"
#include "clang/ASTMatchers/ASTMatchers.h"

using namespace clang::ast_matchers;

namespace clang::tidy::ucassaat {

void DeclareAnonymousStructCheck::registerMatchers(MatchFinder *Finder) {
    // Match struct/union/class definitions that contain fields with record type
    Finder->addMatcher(
        recordDecl(isDefinition(),
                    has(fieldDecl(hasType(recordType())))
                    .bind("field")))
        .bind("record"),
        this);
}

void DeclareAnonymousStructCheck::check(const MatchFinder::MatchResult &Result) {
    const auto *Record = Result.Nodes.getNodeAs<RecordDecl>("record");
    const auto *Field = Result.Nodes.getNodeAs<FieldDecl>("field");

    if (!Record || !Field || !Record->getLocation().isValid() ||
        !Field->getLocation().isValid())
        return;

    // Get the type of the field
    const Type *FieldType = Field->getType().getTypePtrOrNull();
    if (!FieldType)
        return;

    // Check if it's an elaborated type (e.g., 'struct X')
    if (const auto *Elaborated = dyn_cast<ElaboratedType>(FieldType)) {
        FieldType = Elaborated->getNamedType().getTypePtrOrNull();
        if (!FieldType)
            return;
    }

    // Get the record declaration from the field type
    const RecordType *RecordTy = dyn_cast<RecordType>(FieldType);
    if (!RecordTy)
        return;

    const RecordDecl *InnerRecord = RecordTy->getDecl();
    if (!InnerRecord || !InnerRecord->isAnonymousStructOrUnion())
        return;

    // Emit diagnostic
    diag(Field->getLocation(),
         "禁止结构体定义中含有匿名结构体 [gjb8114-r-1-1-9]")
        << Field->getSourceRange();

    // Add note pointing to the outer struct definition
    if (Record->getIdentifer()) {
        diag(Record->getLocation(), "外层结构体 '%0' 定义在此",
            DiagnosticIDs::Note)
            << Record->getName() << Record->getSourceRange();
    } else {
        diag(Record->getLocation(), "外层结构体定义在此", DiagnosticIDs::Note)
            << Record->getSourceRange();
    }
}

} // namespace clang::tidy::ucassaat
```

图 7 输出检查器代码示例